

AMATH 482/582: HOME WORK 2

EUNICE SEUNG-HYUN HAN

Department of Applied Mathematics, University of Washington, Seattle, WA
eunich@uw.edu

ABSTRACT. This study demonstrates the algorithm that will be able to recognize which movement OptimuS-VD is performing in real-time through the built-in sensors that record the movements of the joints. The results indicate that the use of PCA builds a projection of the recordings in lower dimensions while maintaining high accuracy in the prediction and visualizing the movements. Figures show the projected X_{train} in the truncated PCA space in 2D and 3D, which shows the distribution patterns of each movement. Tables show the accuracy of the classifier predicting the train and test labels by their corresponding k modes. This study can be used in the robotics and automation industry as the algorithm predicts the movement of the robot in real-time based on the sensor-recorded joint positions.

1. Introduction and Overview

The next version of the humanoid robot OptimuS-VD is currently in development. It is equipped with built-in sensors that capture the movements of 38 joints at a rate of 60Hz, recording them as Euler angles, which are then transformed into (x, y, z) coordinates. OptimuS-VD is capable of performing three types of movements: walking, jumping, and running. However, given the recorded 38-joint movement data at 60 Hz in Euler angles, we aim to classify the type of movement OptimuS-VD is performing in real-time. To achieve this, we will apply Singular Value Decomposition (SVD) and Principal Component Analysis (PCA) to project the sample data into a lower-dimensional space, enabling visualization of the movements. Based on this projection, we will develop an algorithm capable of recognizing and classifying movements in real-time. This report aims to identify and classify the movement that OptimuS-VD is performing in real-time by using its built-in sensors that record the movements of 38 joints.

2. Theoretical Background

The Singular Value Decomposition is a matrix factorization mathematical technique used in dimensionality reduction. For any $X \in \mathbb{R}^{n \times m}$:

$$(1) \quad X = U \Sigma V^T$$

- X is the X_{train} data that is the matrix of 1500×114
- Orthonormal matrix $U \in \mathbb{R}^{n \times n}$
- Diagonal matrix $\Sigma \in \mathbb{R}^{n \times m}$ with positive entries
- Orthonormal matrix $V \in \mathbb{R}^{m \times m}$

Using SVD, the dimensions of X can be reduced by taking the approximation of the first k columns of each U, Σ, V^T . By keeping the top k singular values and its corresponding singular vectors it gives the rank- k approximation:

$$(2) \quad X_k = U_k \Sigma_k V_k^T$$

This approximation of X , in reduced dimensionality k is projected onto the top k singular vectors. Reducing the dimensions is effective and useful because most information is concentrated in the

first few singular values and it would ignore smaller singular values so it will remove the noise and focus on the most significant data. In large real-world datasets, approximating the top k singular values and their corresponding singular vectors often captures the most essential information. This approach achieves high accuracy while significantly reducing computational and storage costs.

Principal Component Analysis (PCA) [1] is a technique that leverages features from the SVD (1) to reduce the dimension of the data. It identifies the principle components that capture the most important information and variance within the dataset, allowing us to retain a significant amount of information while using less dimensions. Below shows how PCA is implemented using scikit-learn [5] in Python:

$$(3) \quad X_c = U\Sigma V^T$$

$$(4) \quad X_{PCA} = X_c V_k$$

- X_c is the centered data of X
- Modes are the columns of V_k

In terms of using the SVD expansion in Equation 4:

$$(5) \quad X_{PCA} = (U\Sigma V^T)V_k$$

$$(6) \quad = U_k \Sigma_k$$

- Coefficients in PCA-transformed space are given by $U_k \Sigma_k$

The Inverse Transform in PCA:

$$(7) \quad X_{reconstruct} = X_{PCA} V_k^T + X_{mean}$$

Frobenius norm [3] is used in Task 1 and it allows to get the cumulative energy of how many k modes are needed to approximate the X_{train} data up to a certain percentage.

The equation of the Frobenius norm, $\|X\|_F$:

$$(8) \quad \|X\|_F = \left(\sum_{j=1}^{\min(m,n)} \sigma_j^2 \right)^{\frac{1}{2}}$$

The equation of the energy:

$$(9) \quad \|X\|_F^2$$

By calculating the energy ratio using the Frobenius norm, we can determine the most accurate k modes and assess how much variance (information) is retained in the data. In Task 2, truncating the PCA modes uses Equation (4), for $k = 2$ and $k = 3$. In Task 3, the centroid is computed for each movement (jumping, running, and walking) by averaging the corresponding data points after applying the PCA [1]. This centroid helps getting the minimal distance between data points when classifying which movement OptimuS-VD is performing.

3. Algorithm Implementation and Development

In Task 1, the PCA [1] class from scikit-learn [5] was applied to X_{train} , where the PCA modes represent spatial modes and the coefficients are time-dependent coefficients. The PCA [1] class from scikit-learn [5] does the centering of data automatically for obtaining the energy ratio in each k modes. On the other hand, centering the data before applying the PCA [1] is essential in Task 1 because it keeps the mean fixed and ensures consistency for every iteration in the for loop of different k modes that changes the outputs of the PCA [1], its inverse transform (7), and the corresponding energy (9). Within a for-loop that iterates over k modes, the PCA [1] is applied to the centered X_{train} data which produce lower k -dimensional representation. Then, by using the inverse transform of the PCA [1], the data is projected back from the PCA space to the original

space, reconstructing the data using the principal components. The inverse transform [5] allows us to check how much information is lost in a particular k mode and to assess how well the PCA [1] preserves the structure of the original data. Then it computes the energy ratio by dividing the Frobenius norm (8) of inverse transformed data by the Frobenius norm (8) of the centered train data in the original space. When this energy ratio(9) exceeds 70%, 80%, 90%, 95%, it provides the corresponding PCA k spatial modes.

In Task 3 and 4, in order to classify what type of movement OptimuS-VD is performing for each sample in X_{train} , by computing the minimal distance between the projected point in k -modes PCA space and each of the centroids (the mean of each movement's PCA transformed data), each sample is then assigned to either jumping, running, or walking movements. Therefore by comparing the assigned labels with the ground truth labels of X_{train} using the accuracy score command [5], it allows us to determine the accuracy for different k values and to identify the optimal k for the classifier. In Task 5, the PCA transform [1] is applied to the X_{test} data and the accuracy score [5] is computed in the same way as in Task 4. This evaluates how well PCA generalizes to new test data using the train data that is fitted to PCA [1].

The software packages employed are NumPy [3], Scikit-learn [5], and Matplotlib [4]. NumPy [3] does numerical computations such as getting the Frobenius norm. Scikit-learn [5] is a machine learning library that contains centering the data, PCA transformation, and getting the accuracy score between data samples. Matplotlib [4] plots the cumulative energy by PCA k modes and how each movements (jumping, running, and walking) is visualized in the truncated PCA space in 2D and 3D.

4. Computational Results

Task 1:

Approximate X_{train} up to ...	70%	80%	90%	95%
Needed PCA spatial modes	2	3	5	7

TABLE 1. Presents the k modes that approximates X_{train} up to a certain percentage.

Table 1 shows how many PCA [1] spatial modes are needed to approximate X_{train} up to each 70%, 80%, 90%, 95% in Frobenius norm (8). It tells to approximate X_{train} up to 70%, we need 2 modes ($k=2$) in PCA [1] and to approximate up to 80% we need 3 modes, and to approximate up to each 90% and 95%, each 5 modes and 7 modes are needed.

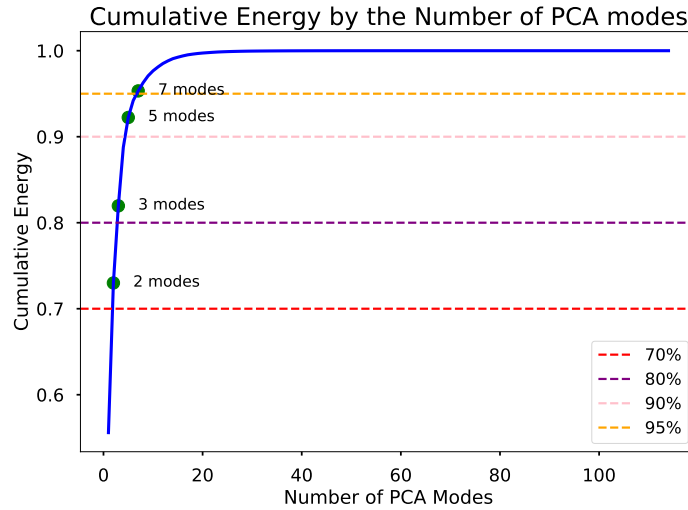
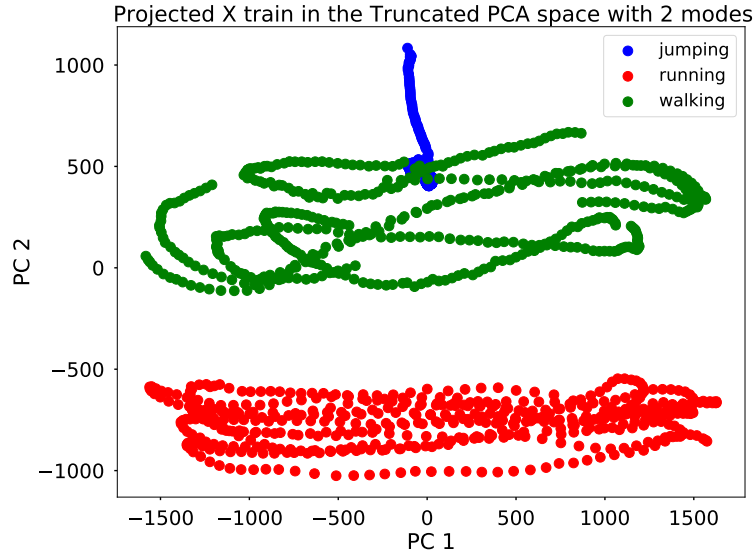


FIGURE 1. The cumulative energy of the X_{train} data in a 2D plot.

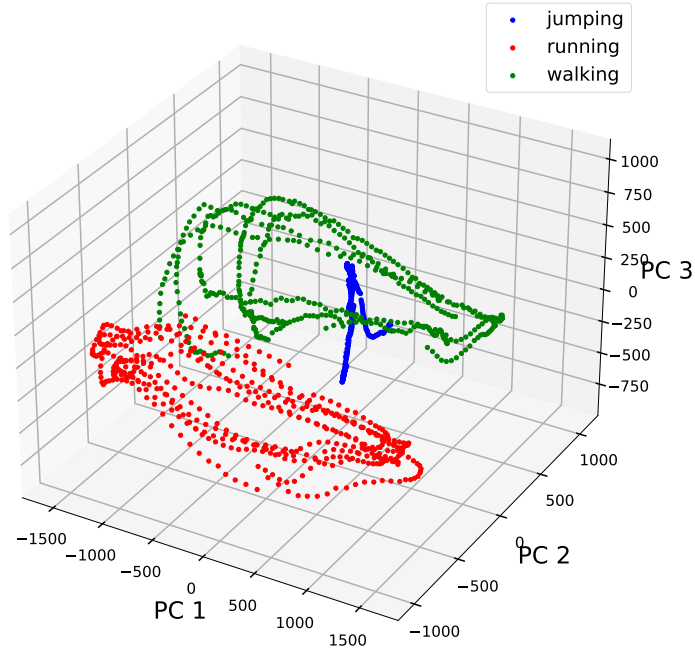
Figure 1 illustrates the cumulative energy of the X_{train} data by the values of k -modes in PCA. It shows the points where the cumulative energy exceeds each 70%, 80%, 90%, 95%, corresponding to the k values listed in Table 1. It demonstrates how k modes approximate the train samples by reducing the dimensions to k , represented as X_{train_k} .

Task 2:



(A)

Projected X train in the Truncated PCA space with 3 modes



(B)

FIGURE 2. The projected X_{train} in the truncated PCA space in 2D and 3D.

Figure 2a shows the projected X_{train} in the truncated PCA space as low dimensional 2D and Figure 2b shows it in low dimensional 3D for each movements; jumping, running, and walking. These visualizations illustrate the distribution patterns of each movement and features how their representations are shaped differently within the truncated PCA space.

Task 3:

k -modes	1	2	3
jumping	-23.89	499.37	-72.50
running	60.77	-752.72	-103.41
walking	-36.88	253.35	175.91

TABLE 2. Sample Centroids in k -modes PCA space.

The centroid changes in every different k -modes that are used in Task 4 and Task 5. It is utilized to compute the minimal distance between the projected point in k -modes PCA space and its corresponding centroid. This eventually provides the accuracy of the trained classifier in the prediction of the train and test data.

Task 4:

k values of k -PCA truncation	1	2	3	4	5	10	14	20
Accuracy of the trained classifier	50.73%	88.13%	75.6%	73%	75.07%	88.8%	91.07%	91.07%

TABLE 3. Accuracy of the trained classifier by its corresponding k modes.

Table 3 shows how the selection of k modes impact the accuracy [5] of the trained classifier. This represents the percentage of samples where the ground truth labels and the predicted trained labels match. The results indicate that the optimal k value is 14 modes for the classifier to achieve the highest accuracy percentage: 91.07%.

Task 5:

k values of k -PCA truncation	1	2	3	4	5	10	11	14
Accuracy predicting test labels	48.67%	98.33%	92.33%	74.67%	91.67%	94.33%	95.33%	95.33%

TABLE 4. Accuracy of the trained classifier predicting the test labels by its corresponding k modes.

Table 4 shows the results of how the trained classifier performs on classifying the new sample sets. By projecting the data onto the k -PCA space and computing the minimal distance to the centroids, the accuracy of the prediction of the test labels is presented in Table 4. By computing using multiple k values (as shown in Table 4), the highest accuracy was achieved when $k = 11$, telling the optimal k value is 11 modes with the accuracy reaching up to 95.33%. Comparing with the train data accuracy from Table 3, the accuracy for both exceeded over 90% which tells the classifier has performed proficiently and effectively on both train and test data samples.

5. Summary and Conclusions

The goal of this study was to predict the movements - jumping, running, and walking - of the next version of the humanoid robot OptimuS-VD that has built-in sensors that record the movements at 60Hz as Euler angles that transforms into (x, y, z) coordinates. By using Principal Component Analysis (PCA) [1], the input data achieves high approximation with lower dimensionality. Figure 2a and 2b shows the projected X_{train} in the truncated PCA space as low dimensional 2D and 3D. This visualization shows what formation and pattern of each performance of the movement. The classifier predicts the most accurate movement of OptimuS-VD by computing the minimal distance between the data point and the centroid of its corresponding k modes in PCA. This classifier has demonstrated high accuracy rate as shown in Table 3 and 4.

Furthermore, this study can be extended to develop a system that will recognize which movement OptimuS-VD is performing in real-time by using the data of the movements and location of its joints that are recorded through the built-in sensors as Euler angles. In the future, this study could contribute valuable and enhanced resources to the robotics and automation industry as the algorithm would predict the movement of the robot in real-time based on sensor recorded joint positions.

Bonus Problem:

The alternative classifier that is implemented in the k -PCA [1] space is the KNeighborsClassifier from scikit-learn [5]. KNeighborsClassifier is a machine learning algorithm that does classification. It stores the entire training dataset and makes prediction based on the nearest neighbors of that given data point. It calculates the distance between the data point and its surrounding neighbors with selecting n neighbor points and predicts the label based on the majority class among those neighbors.

k values in k -PCA space	1	2	3	4	5	10	12
Accuracy of the train data	73.13%	98.8%	99.2%	99.6%	99.6%	100%	100%
Accuracy of the test data	70.67%	100%	99.33%	97.67%	98.33%	100%	100%

TABLE 5. Accuracy of the train and test data using KNeighborsClassifier.

The fixed n neighbors is 10 for computing the accuracy of the train and test samples in Table 5. The results indicate that KNeighborsClassifier presents higher accuracy compared to the results above where we computed accuracy based on the minimal distance between data points and their centroid. The key difference between the two methods is KNeighborClassifier can handle nonlinear data and make predictions based on the local patterns of the sample data. In contrast, the centroid-based classification approach relies on the global average of the dataset. Since the data contains overlapping or nonlinear data distributions, that could be why KNeighborClassifier shows higher accuracy.

ACKNOWLEDGEMENTS

The author is thankful to the TA Rohin Gilman for going over the general information of the homework and explaining the mathematical concept of SVD and how it is related to PCA which helped understanding the given tasks and commands in codes when doing the assignment.

REFERENCES

- [1] PCA.
 - [2] N. Frank. Amath 482 A Instructor Notes. Lecture 9, 10: Singular Value Decomposition(SVD).
 - [3] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, Sept. 2020.
 - [4] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
 - [5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of machine learning research*, 12(Oct):2825–2830, 2011.
- [1][2][3][4][5]